

Financial Analysis and Risk Auditing Agent Report

1. Problem Statement

This agent functions as a financial analyst that evaluates public stocks and cryptocurrencies to produce a risk-adjusted investment recommendation. The problem naturally benefits from multi-step chaining because large language models are not reliable at deterministic mathematical computation and often struggle to critically evaluate their own reasoning within a single prompt. If the entire workflow were handled through one monolithic prompt, the model would likely hallucinate technical indicators and generate overly confident or overly agreeable recommendations. By decomposing the task into multiple stages, the architecture separates deterministic numerical computation from language-based reasoning and introduces an adversarial auditing mechanism where recommendation generation and critique are handled independently. This results in a more reliable, explainable, and evidence-driven final output.

2. Chain Design

The entire pipeline is coordinated through a shared Python dictionary representing the global agent state. Each step reads only the fields it requires and appends its outputs back into the state object. This allows downstream stages to trace the full reasoning history while keeping each step modular and independently testable.

The agent is built as a structured eight-step pipeline in which every stage has a narrowly defined responsibility and operates within strict cognitive boundaries. The pipeline begins with the first LLM call, which parses the raw user query to identify and validate the requested financial asset while extracting a canonical ticker symbol. This stage acts as a gatekeeper for the rest of the system, ensuring that downstream API calls are not wasted on malformed or ambiguous inputs.

Once the asset has been validated, the pipeline transitions into deterministic processing through external tools. The first tool call retrieves historical market data using APIs such as Yahoo Finance for equities and CoinGecko for cryptocurrencies. This is followed by the second tool call, where pure Python computations are used to calculate technical indicators such as moving averages, volatility measures, RSI values, and trend signals from the retrieved price history. These stages are intentionally implemented as tool calls rather than LLM calls because mathematical computation must remain deterministic and reproducible.

After the quantitative analysis is completed, the pipeline gathers qualitative market context through the third tool call, which retrieves recent financial news headlines using a Google News RSS feed. The headlines are then passed into the second LLM call, where the model performs sentiment analysis and generates a sentiment score along with a brief explanation of the market tone. This stage is isolated from the technical indicators so that the model focuses exclusively on natural language interpretation rather than mixing qualitative and quantitative reasoning simultaneously.

The outputs from the numerical and sentiment stages are then combined in the third LLM call, which acts as a reconciliation layer. At this stage, the model evaluates whether the technical indicators, market data, and sentiment signals support or contradict one another, producing a reconciliation score that measures the alignment between quantitative and qualitative evidence.

Using this reconciled evidence, the fourth LLM call generates a preliminary investment recommendation containing a buy, hold, or sell decision along with supporting reasoning and a confidence score. Instead of finalizing the recommendation immediately, the system introduces an adversarial auditing phase through the fifth LLM call. In this stage, the model acts as a risk auditor that stress-tests the logic of the draft recommendation, identifies unsupported assumptions or contradictory evidence, and proposes a confidence penalty if the recommendation

appears overly certain.

Finally, the pipeline concludes with the sixth LLM call, which serves as a senior synthesis layer. This stage receives both the original recommendation and the critique produced by the auditor, reconciles the two perspectives, adjusts the confidence level if necessary, and produces the final structured investment recommendation.

3. Tool Integration

The pipeline integrates three external tools: the Yahoo Finance API for stock data, the CoinGecko API for cryptocurrency data, and the Google News RSS feed for recent financial news. In addition, the system uses a custom Python-based computation module that calculates technical indicators such as moving averages, RSI, volatility, and trend signals from raw historical price data. These tools are necessary because the language model does not have reliable access to real-time market prices or deterministic mathematical computation. The retrieved outputs are cleaned, converted into structured JSON, and stored in the shared system state. For example, raw price data fetched through Yahoo Finance is processed by the custom Python computation stage to generate technical indicators, which are later provided to the reconciliation LLM as structured evidence.

4. Limitations

The primary limitation of this pipeline is its dependence on accurate asset identification in the first LLM stage. If a user provides a vague query, the model may extract the wrong ticker symbol, causing the downstream analysis to fail. The system also struggles with newly listed stocks or cryptocurrencies introduced after the model's knowledge cutoff, since the LLM may not recognize them correctly. Another limitation is the sentiment analysis stage, which depends heavily on the quality of the retrieved news headlines. If the RSS feed contains speculative or opinion-based articles instead of factual financial reports, the final recommendation can become skewed. A future improvement would be adding a secondary verification tool to validate ticker symbols against a live financial database before continuing the pipeline.

5. Reflection

During development, I found an interesting insight regarding positional framing while discussing prompt design ideas with ChatGPT. I noticed that explicitly telling the model "You are Step X of a pipeline" seemed to keep each stage focused on its assigned responsibility, which made me curious about what would happen if this framing was removed entirely. After testing this experimentally, I observed that the model often tried to do more than it was supposed to during intermediate stages. Some steps attempted to generate full investment recommendations instead of performing their intended subtasks, while others hallucinated additional analysis or skipped over raw factual processing by producing high-level summaries instead of structured outputs. This experiment showed that positional framing acts as an important behavioral constraint that improves reliability and keeps each stage focused on its specific role within the pipeline.

If I had more time, I would implement a dynamic retry mechanism where the final synthesis stage could send the recommendation back for a complete rewrite if the critique from the auditing stage was too severe. I also used a large language model during development to help debug JSON extraction logic and brainstorm adversarial critique prompts. However, the initial prompts produced vague and hallucinated market risks, which required substantial manual refinement before they became reliable.

Appendix: Prompt Design Explanation

Every step in this chain uses JSON prompts, with each step receiving only the specific subset of the global state it actually needs. This keeps the model's context focused and manageable. The system prompts also enforce strict type constraints — for example, requiring confidence scores to always be integers between 0 and 100 — because downstream steps rely on predictable key-value structures to operate reliably without breaking.

The Evolution of Step 7 (Prompt Iteration)

Designing the self-critique mechanism in Step 7 took several iterations. My initial approach used an “Adversarial Debater” persona that explicitly forced the model to argue against the draft recommendation.

Version 1 (Failed Prompt):

“You are Step 7, a critical analyst reviewing a draft recommendation... Your job is to argue for the opposite position. Identify every weakness in the draft's reasoning...”

Why it failed:

Because the model was required to take the opposite stance, it often invented weak or speculative criticisms even when the underlying data strongly supported the recommendation. It would generate generic concerns like “market volatility” simply to satisfy the prompt's requirement to attack the thesis.

Version 2 (Successful Prompt):

I redesigned the role into a “Ruthless Risk Auditor.”

“You are Step 7, a ruthless risk auditor reviewing a draft recommendation... Your job is to assess whether the stated confidence is justified by the evidence. Ask: are there real data gaps, contradicting signals, or weak evidence... If the signals are strongly aligned and the evidence is solid, say so — a minor critique with a low penalty (0–5 pts) is a completely valid outcome.”

Why it worked better:

This version removed the forced contrarian behavior and instead aligned the model toward evidence-based evaluation. The prompt no longer required the model to “find something wrong” with every draft. Instead, it instructed the model to check whether the confidence level was actually supported by the available signals and data quality. By explicitly discouraging generic criticisms like “market uncertainty” and permitting a 0-point penalty when the evidence was genuinely strong, the critique stage became far more calibrated. Rather than acting as a system that automatically opposed the previous step, it started functioning as a true risk validation layer that only applied penalties when there were meaningful weaknesses, missing information, or conflicting signals in the analysis.